Advanced Agent Skills for Power BI

A field guide to teaching a coding agent how your team builds semantic models, and keeping it to that.

For Power BI developers who already use a coding agent with a Power BI MCP server, or are about to. Companion to the session at the Data & AI Builders Conference, Bergamo, 7 November 2026. Mathias Halkjær.

New to skills?
Read chapter 1, about 15 minutes. It builds the idea one piece at a time.

Ready to build one?
Chapter 2, starting with "Set up once".

Skill not being used?
Go to 2.3, "Check that it's used".

Need a template?
The one-page card, 3.1.

1 How it works

1.1 The problem

1.2 A skill

1.3 How a skill is found

1.4 Reference files

1.5 Scripts

1.6 Hooks

1.7 Sub-agents

1.8 A skill that learns

1.9 The whole picture

2 Doing it

2.1 Set up once

2.2 Write your first skill

2.3 Check that it's used

2.4 Add a check script

2.5 Add a guardrail

2.6 Get a model ready for AI

2.7 Let the skill learn

3 Look it up

3.1 The one-page card

3.2 What works where

3.3 Before you rely on it

3.4 Words

CHAPTER 1

How it works

The idea, built one piece at a time. Sections 1.2 to 1.8 each add one of seven pieces, shown in a strip at the top of each section: pieces you have are grey, the new one is dark, the ones to come are dashed. In the figures, an outline in the accent colour marks the piece the section is about. 1.9 puts the pieces together.

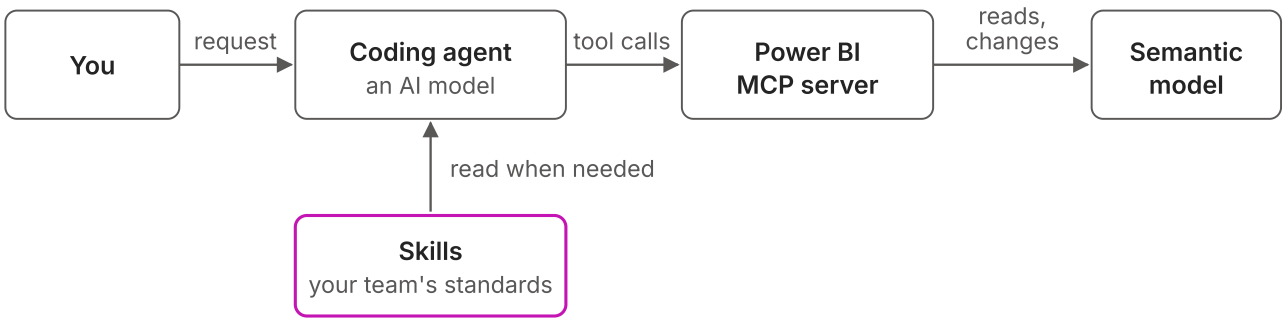
1.1 The problem
1.2 A skill
1.3 How a skill is found
1.4 Reference files
1.5 Scripts
1.6 Hooks
1.7 Sub-agents
1.8 A skill that learns
1.9 The whole picture

1.1

The problem

A coding agent connected to your semantic model can rename fifty columns in a minute. Left to itself, it renames them its own way.

The agent in this guide is a **coding agent**: an AI assistant that can read files, run commands and call tools. You chat with it in an **agent harness**, or **harness** for short: the app that runs the model, reads the skills and calls the tools, such as Claude Code, GitHub Copilot in VS Code or Cursor. The examples use Claude Code, which runs in a terminal or inside VS Code. Connected to a **Power BI MCP server**, the agent can read and change a semantic model and run DAX queries against it.



How a request travels. The agent knows Power BI in general; the skills below it are where your team's own way of working comes in.

The agent knows DAX and Power BI well. What it doesn't know is your naming standard, your DAX style, which workspace is production, or that the finance team says "net sales" and never "revenue". And every new conversation starts from zero, so whatever you explained yesterday is gone today.

THINK OF IT LIKE

A skilled consultant on their first day. They know Power BI, but not how your team does things. You give them the team handbook, and they look up the right page when they need it. A skill is a page of that handbook, written for the agent.

WORDS YOU'LL MEET

MCP server: a small program that gives the agent tools, here tools for a semantic model. MCP (Model Context Protocol) is the open standard most coding agents use to connect to tools.

PBIP and TMDL: saving a Power BI project as PBIP turns the semantic model into text files in the TMDL format, one file per table. The agent can read and change them like any other files, and git can track every change.

Working memory: everything the agent has read in the current conversation. It's limited, and everything in it competes for the agent's attention. Technically it's called the context window.

TWO WAYS TO THE MODEL

The agent can reach a semantic model in two ways: through the MCP server, which works on a model open in Power BI Desktop or published to a workspace, and through the TMDL files of a PBIP project, which it edits like any other text files. This guide makes the changes in the TMDL files, where git shows every one of them, and uses the MCP server to look things up and run DAX queries. Power BI Desktop, and an MCP server connected to it, see changes to the files when you reopen the project.

The files are your working copy. Production is the model published in a workspace, and the agent can only reach it through the MCP server. That's where the guardrail in 1.6 sits.

The running example throughout is the **Sales model**, which the team wants ready for AI: Copilot and Fabric data agents answer better when tables, columns and measures have clear names, descriptions and display folders. It's real work, it's repetitive, and it has to follow the standard everywhere, which makes it a good job for an agent with the right skill.

1.2

THE IDEA SO FAR **Skill** | Description | References | Scripts | Hooks | Sub-agents | Learning

A skill is a folder with one required file

A **skill** is a folder with a file called SKILL.md in it. The file holds a name, a description of when to use the skill, and instructions in plain text.

```
---
name: model-standards                                    ← the skill's name
description: Naming, display folder and DAX
  conventions for our semantic models. Use when          ← when to use it (1.3)
  creating, renaming or documenting tables,
  columns or measures, or when writing DAX.
---
# Model standards

## Names
- Tables: singular business nouns, no prefixes          ← plain instructions
  (Customer, not DimCustomer).
- Columns and measures: words with spaces,
  first letter capital (Order Date).
## DAX
For style and patterns, read                             ← a reference file (1.4)
references/dax-style.md.
## Before you finish
Run scripts/check_names.py and fix                       ← a script (1.5)
what it reports.
```

The SKILL.md of model-standards, shortened. The part between the two "---" lines is the header; the rest is instructions, written the way you'd brief a colleague.

The folder is named after the skill and can hold more than SKILL.md. Sections 1.4 and 1.5 cover the other two parts:

```
.claude/skills/model-standards/
  SKILL.md          required: header and instructions
  references/       longer material, read when needed (1.4)
  scripts/          small programs for exact checks (1.5)
```

Agent Skills is an open standard that several harnesses support, so the same folder works in more than one of them. Where you put it differs per tool. The examples in this guide use Claude Code's locations: `.claude/skills/` in the project for skills the team shares, and `.claude/skills/` in your

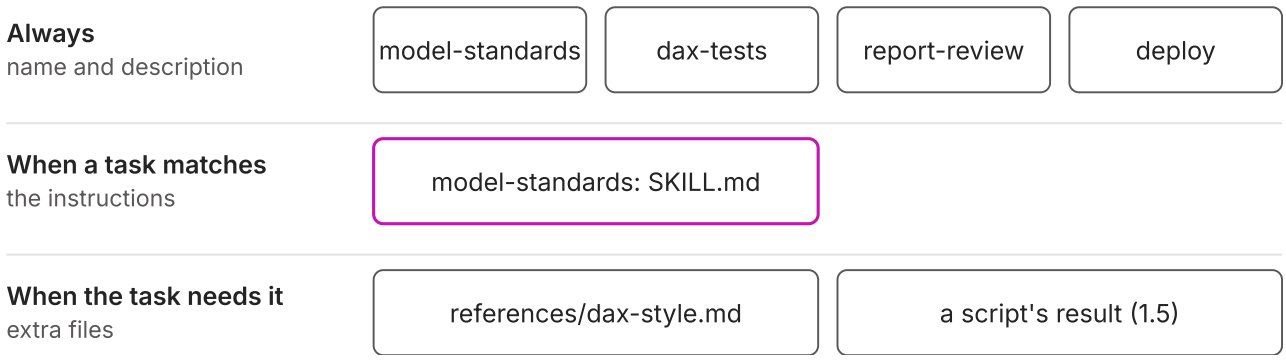
user folder (on Windows, `C:\Users\<you>\.claude\skills\`) for your personal ones. A project skill sits in git next to the PBIP files, so everyone on the team gets the same standards, and changes to them are reviewed like any other change.

1.3

THE IDEA SO FAR Skill **Description** References Scripts Hooks Sub-agents Learning

The description decides when a skill is used

At the start of a conversation the agent sees only each skill's name and description. It reads the instructions only when a task matches the description.



What the agent reads, and when. Every skill's description is always in view; the rest is read only when a task calls for it.

Whether a task matches is the agent's own judgement, made by reading the descriptions, not a keyword search. This is why you can install many skills without filling the agent's working memory: each one costs a couple of lines until it's needed. It also makes the description the most important line in the skill. It has to say what the skill is for and when to use it, in the words people actually type: "rename", "measure", "display folder", "DAX".

A vague description such as "Helps with Power BI" fails both ways. The agent either never connects it to a task, or reaches for it on every task. When a skill isn't used, the description is almost always the reason; 2.3 shows how to test it.

You can also call a skill by name. In Claude Code, typing `/model-standards` in the chat loads it directly.

> WHY NOT ONE BIG INSTRUCTION FILE?
>
> Coding agents also read a project file at the start of every conversation (CLAUDE.md in Claude Code, AGENTS.md in several others). It's the right place for the few things every task needs. Put your whole handbook there and it's read for every task, including the many that don't need it, and it crowds out the work.

1.4

THE IDEA SO FAR Skill | Description | References | Scripts | Hooks | Sub-agents | Learning

Reference files: longer material, read only when needed

Material that only some tasks need goes in separate files that SKILL.md points to. The agent opens a file only when the task calls for it.

model-standards has two reference files: `dax-style.md`, the team's DAX formatting and patterns, and `glossary.md`, the business terms with the words people use for them. SKILL.md says when to read each one, in a sentence: "For DAX style and patterns, read references/dax-style.md."

"Rename the Customer columns" a naming task	**"Fix the Margin % measure"** a DAX task	**"Describe every Sales column"** a documentation task
SKILL.md	SKILL.md	SKILL.md
	dax-style.md	glossary.md

Same skill, three tasks. Each task reads SKILL.md, plus only the reference file it needs.

A good rule: SKILL.md holds what every task under the skill needs, and stays short, a few hundred lines at most. Everything else goes in a reference file, one topic per file, linked directly from SKILL.md rather than from another reference file, so the agent always knows where to look.

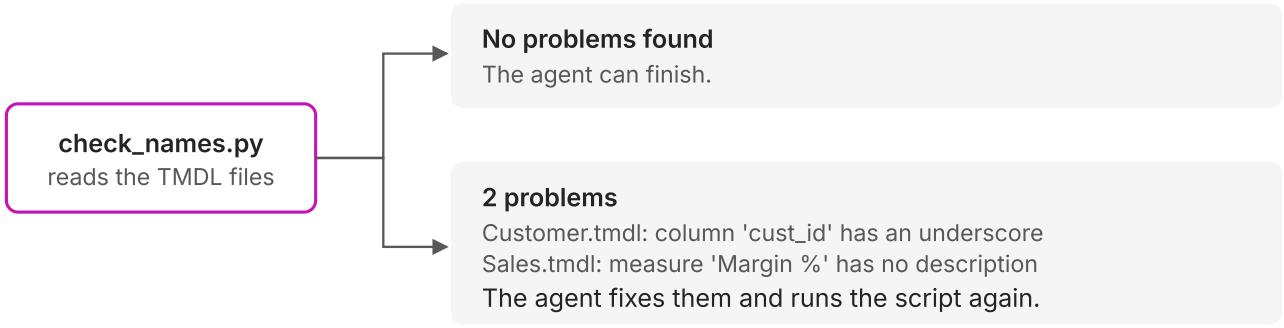
1.5

THE IDEA SO FAR Skill | Description | References | **Scripts** | Hooks | Sub-agents | Learning

Scripts: checks that don't depend on judgement

Some checks shouldn't be left to the agent. A script in the skill does them the same way every time, and the agent reads only the result.

model-standards has a script, `check_names.py`, that reads the model's TMDL files and lists every name that breaks the standard and every measure without a description. The skill tells the agent to run it before finishing.



One script, two outcomes. Only the short result enters the agent's working memory, never the files the script read.

An agent reading 400 column names will miss a few. A script won't, and it costs the agent almost nothing: the script reads the files, and the agent reads a short list. The script here is a small Python program, so Python has to be installed; 2.4 has the whole script. Anything you can check exactly, such as formatting rules, is a candidate for a script.

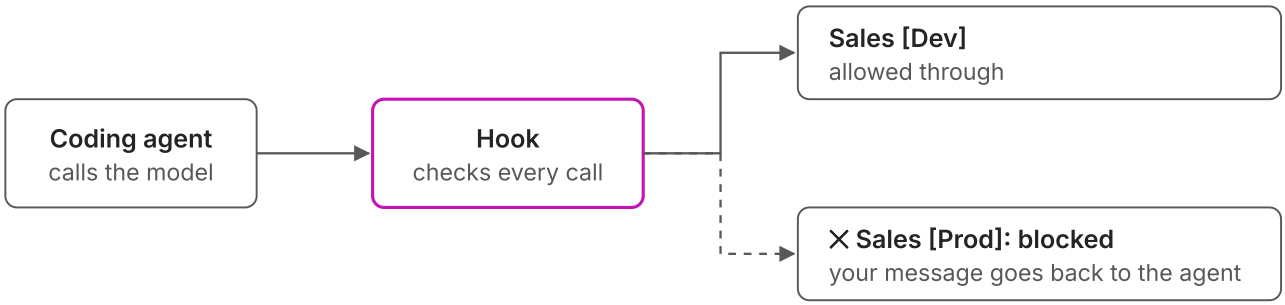
A script is still something the agent *chooses* to run, because the instructions ask it to. Usually that's enough. When it must happen every time, without exception, you need the next piece.

1.6

THE IDEA SO FAR Skill | Description | References | Scripts | **Hooks** | Sub-agents | Learning

Hooks: guardrails the agent can't skip

A **hook** is a command the harness runs automatically before or after an action. The agent can't skip it, because the agent isn't the one running it.



A guardrail on every call to the model. The dashed line is a call that never arrives.

Two hooks that fit Power BI work well:

- **Before** any call to the Power BI MCP server, block it if it's aimed at the production model, the only way the agent can reach production. The agent gets your message, "Sales [Prod] is production; work on Sales [Dev]", and carries on there.
- **After** any file change, run check_names.py and give the agent the result. It runs after every file edit, checks only the file that was just edited, and ignores anything that isn't TMDL. Now the hook runs the check from 1.5 after every change; the line in SKILL.md stays as a backup for harnesses without hooks.

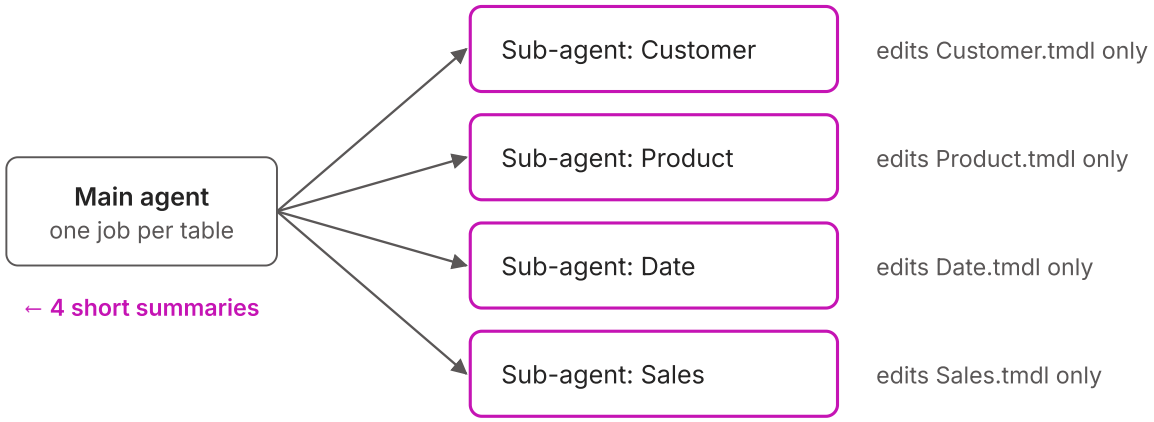
WRITTEN AS	EXAMPLE	WHAT HAPPENS
An instruction in a skill	"Never change the production model."	The agent follows it almost always.
A hook	Block calls aimed at Sales [Prod].	Enforced every time.

Use both: the instruction tells the agent why, so it doesn't keep trying, and the hook makes sure. Hooks aren't part of the open skills standard. They're a feature of the harness, set up in its settings files; 2.5 shows Claude Code's version.

Sub-agents: split the work, keep the main conversation clear

A **sub-agent** is a second agent with its own, empty working memory. The main agent gives it one job and gets back a short answer.

Empty means it starts without the main conversation, not without skills: when you describe a sub-agent, you can give it the skills it needs (2.6), so every sub-agent works to the same standard.



One job per table, in parallel. Each sub-agent sends back a three-line summary, so the main agent's working memory holds four summaries, not four tables' worth of detail.

There are two ways to use them with skills:

- **Fan out.** To get the Sales model ready for AI, one sub-agent per table writes descriptions and sets display folders, all at the same time, each following model-standards. Because PBIP keeps each table in its own TMDL file, they never edit the same file.
- **Run a skill as a sub-agent.** A skill that reads every measure in the model produces a lot of detail you don't need afterwards. Run as a sub-agent, that reading happens out of the main conversation, and only the conclusion comes back. In Claude Code, the line `context: fork` in the skill's header does this.

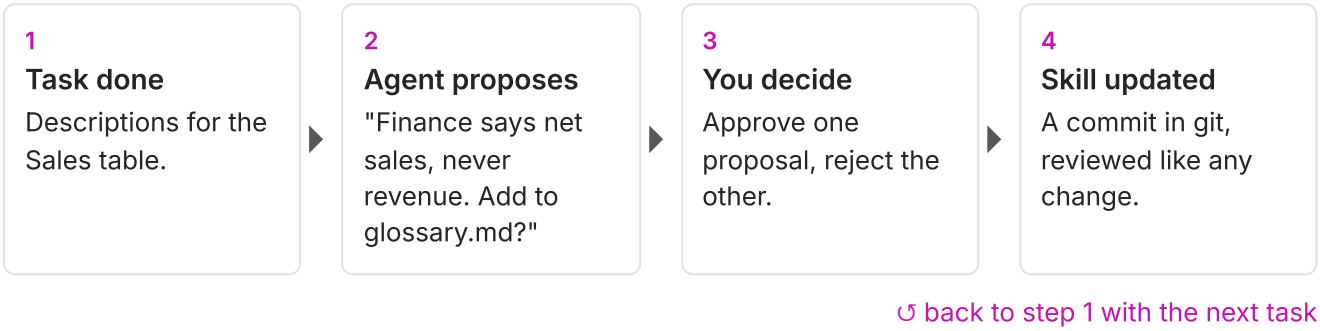
Fanning out helps when the pieces are independent, like tables getting descriptions. It hurts when they depend on each other, like measures that reference other measures, or relationships between tables. Then several agents working at once just make the same mistake several times, faster. Do that work in one place.

1.8

THE IDEA SO FAR Skill | Description | References | Scripts | Hooks | Sub-agents | **Learning**

A skill that learns, with you deciding

At the end of a task, the agent proposes changes to the skill: a convention it met that isn't written down, or a mistake it shouldn't make again. You decide what goes in.



The learning loop. The next task starts with a slightly better skill, and every change to it can be traced and undone.

It takes no special feature, only a section in SKILL.md: "After each task, list conventions you met that aren't in this skill, and mistakes you corrected. Propose additions to the right reference file. Don't change the skill until I approve."

The same idea gives you documentation as you go. A file such as `references/decisions.md` collects decisions with the reason for each, such as "Net Sales excludes returns, agreed with finance on 3 November". The agent adds to it when a decision is made and reads it when the question comes up again.

> WHY A PERSON DECIDES
>
> A skill that rewrites itself unattended drifts. One wrong lesson, such as a rule learned from a one-off exception, then gets applied to every model the team builds. Keeping the skill in git and approving each change keeps it the team's standard, not the agent's.

1.9

The whole picture

One request, "get the Sales model ready for AI", uses every piece. Here is where each one comes in.

STEP	WHAT HAPPENS	PIECE
1	The request matches the description of model-standards, so its SKILL.md is read.	Skill, description
2	The task is documentation, so the agent also opens glossary.md.	References
3	The main agent gives one table to each sub-agent.	Sub-agents
4	Every call through the MCP server passes the hook; production is blocked.	Hooks
5	After each change, a hook runs check_names.py, and the agent fixes what it lists.	Scripts, hooks
6	The agent proposes two additions to the skill; you approve one.	Learning

One request, from start to finish. The agent decides on steps 1, 2, 3 and 6; the harness runs the hooks in steps 4 and 5 regardless.

The seven pieces, in one line each

1. **Skill**: a folder with SKILL.md, your instructions in plain text.
2. **Description**: always visible to the agent, and it decides when the skill is used.
3. **References**: longer material, read only by tasks that need it.
4. **Scripts**: exact checks; the agent reads only the result.
5. **Hooks**: guardrails the harness runs on every action, which the agent can't skip.
6. **Sub-agents**: separate working memory for separate jobs, in parallel when they're independent.
7. **Learning**: the agent proposes changes to the skill, and you approve them.

That's the whole idea. Chapter 2 builds it, one task at a time.

CHAPTER 2

Doing it

Seven tasks, in the order you'd do them, all on the Sales model. Each has the same shape: when to use it, what you need, the steps, and how you know it's done. 2.1 is for whoever sets up the tools; everyone else can start at 2.2.

2.1 Set up once
2.2 Write your first skill
2.3 Check that it's used
2.4 Add a check script
2.5 Add a guardrail
2.6 Get a model ready for AI
2.7 Let the skill learn

2.1

Set up once

USE THIS WHEN

You're starting, or a new colleague joins the project.

YOU NEED

VS Code with a coding agent (the examples use Claude Code), Power BI Desktop, git, and Python for the scripts.

STEPS

1. Save the report as a Power BI project: in Power BI Desktop, **File** → **Save as**, choose the **.pbip** type, with the TMDL format switched on. The model now lives in `Sales.SemanticModel/definition/`, one TMDL file per table.
2. Put the project folder in git, so every change the agent makes can be reviewed and undone.
3. Add a Power BI MCP server to the coding agent, for example Microsoft's Power BI modelling MCP server, following its own setup guide. You give the server a name when you add it (in Claude Code, the name in `claude mcp add <name> …`); note it, because the hook in 2.5 uses it.
4. Create the skills folder in the project: `.claude/skills/`.

DONE WHEN

You ask the agent "Which tables are in the Sales model?" and it answers by calling the MCP server, not by guessing from file names.

2.2

Write your first skill

USE THIS WHEN

You catch yourself explaining the same standard to the agent twice.

YOU NEED

The project from 2.1, and your team's standard, even if it's only in people's heads.

STEPS

1. Create `.claude/skills/model-standards/SKILL.md`.
2. Write the header: a name in lowercase with hyphens, and a description that says what the skill covers and when to use it, with the words people type (the template is in 3.1).
3. Write five to ten rules, the ones you'd tell a new colleague on day one. Give an example with each rule; "Customer, not DimCustomer" teaches more than "no prefixes".
4. Move anything long, such as the DAX style guide, into `references/`, and start `references/glossary.md` with the business terms and the words people use for them (2.6 needs it). Add one sentence to SKILL.md for each file, saying when to read it.
5. Commit the skill to git.

DONE WHEN

The checks in 2.3 pass.

2.3

Check that it's used

USE THIS WHEN

You've written or changed a description, or a skill isn't being used when you expect it.

YOU NEED

A few requests that should use the skill, and a few that shouldn't.

STEPS

1. Write down five requests that should use the skill and three that shouldn't, in the words people really use:

REQUEST	SHOULD USE MODEL-STANDARDS?
"Rename the columns in Customer to our standard"	✓ yes
"Add a measure for margin percentage"	✓ yes
"Put the date measures in a display folder"	✓ yes
"Tidy up the column names in Product"	✓ yes
"Write descriptions for the Sales measures"	✓ yes
"Why is this report page slow?"	✗ no
"Write a commit message for these changes"	✗ no
"Set up the gateway for scheduled refresh"	✗ no

2. Try each request in a new conversation, and watch whether the agent reads the skill. Claude Code says so in the chat when it loads a skill.
3. If a "yes" request didn't use it, add that request's words to the description. If a "no" request did, say more precisely what the skill is for.

DONE WHEN

All the "yes" requests use the skill and none of the "no" requests do.

2.4

Add a check script

USE THIS WHEN

A rule can be checked exactly, and the agent sometimes misses it.

YOU NEED

The skill from 2.2, and Python.

STEPS

1. Save the script as `.claude/skills/model-standards/scripts/check_names.py`. It checks two rules: no underscores in names, and a description (a `///` line above it in TMDL) on every measure.

```
import json, pathlib, re, sys

def check(file):
    problems = []
    lines = file.read_text(encoding="utf-8").splitlines()
    for i, line in enumerate(lines):
        m = re.match(r"\s*(table|column|measure)\s+('([^']+)'|[^\s=]+)", line)
        if not m:
            continue
        kind, name = m.group(1), m.group(3) or m.group(2)
        if "_" in name:
            problems.append(f"{file.name}: {kind} '{name}' has an underscore")
        if kind == "measure" and not lines[i - 1].strip().startswith("///"):
            problems.append(f"{file.name}: measure '{name}' has no description")
    return problems

if "--hook" in sys.argv:  # run by a hook (2.5): check only the file just edited
    edited = json.load(sys.stdin).get("tool_input", {}).get("file_path", "")
    if edited.endswith(".tmdl") and pathlib.Path(edited).exists():
        problems = check(pathlib.Path(edited))
        if problems:
            print("\n".join(problems), file=sys.stderr)  # shown to the agent
            sys.exit(2)
    sys.exit(0)

folders = [pathlib.Path(a) for a in sys.argv[1:]]
folders = folders or list(pathlib.Path(".").glob("*.SemanticModel"))
problems = [p for folder in folders
            for f in sorted(folder.rglob("*.tmdl")) for p in check(f)]
print("\n".join(problems) or "No problems found")
```

2. Run it once yourself from the project folder: `python .claude/skills/model-standards/scripts/check_names.py`. Expect a list on an existing model; that's the work ahead. Then add a section to SKILL.md: "Before you finish, run scripts/check_names.py and fix everything it reports."

DONE WHEN

You give a column a name with an underscore, ask the agent to tidy the table, and it runs the script and fixes the name without being told.

2.5

Add a guardrail

USE THIS WHEN

A rule must hold every time, such as never touching production. Claude Code's version is shown; other harnesses have their own.

YOU NEED

The MCP server's name from 2.1 (here `powerbi-modeling`) and the name that identifies the production model in the agent's calls (here `Sales [Prod]`). To see how your server names models, ask the agent to connect to Sales [Dev] and look at the call in the chat; production is named the same way.

STEPS

1. Save the guardrail as `.claude/hooks/no_production.py`. It reads the call the agent is about to make, and blocks it if the production model's name is in it. If your MCP server identifies models by an ID, check for the ID instead:

```
import json, sys

call = json.load(sys.stdin)
if "Sales [Prod]" in json.dumps(call.get("tool_input", {})):
    print("Blocked: Sales [Prod] is production. Work on Sales [Dev].",
          file=sys.stderr)
    sys.exit(2)  # 2 means: block the call, show the message to the agent
```

2. Register both hooks in `.claude/settings.json`, the guardrail before every call to the MCP server, and the name check after every file edit. Claude Code names an MCP server's tools `mcp__<server>__<tool>`, so the first matcher catches every tool of that server. Commit both files, so the guardrail applies to everyone on the project:

```
{
  "hooks": {
    "PreToolUse": [{ "matcher": "mcp__powerbi-modeling__.*",
      "hooks": [{ "type": "command", "command": "python .claude/hooks/no_production.py" }]
}],
    "PostToolUse": [{ "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command":
        "python .claude/skills/model-standards/scripts/check_names.py --hook" }] }]
  }
}
```

DONE WHEN

You ask the agent to connect to Sales [Prod], and it's stopped with your message and offers to work on Sales [Dev] instead.

2.6

Get a model ready for AI, with sub-agents

USE THIS WHEN

Many tables need the same independent work, here descriptions and display folders, so Copilot and data agents understand the model.

YOU NEED

The skill with glossary.md, the script, and the guardrail from 2.2 to 2.5.

STEPS

1. Describe the sub-agent in `.claude/agents/table-documenter.md`. The `skills` line gives it model-standards from the start:

```
---
name: table-documenter
description: Documents one table of a Power BI semantic model.
tools: Read, Edit, Grep, Glob
skills: model-standards
---
You document one table of a PBIP semantic model. Edit only that
table's TMDL file. Give every column and measure a one-sentence
description in business words from glossary.md, and a display
folder. Reply in three lines: what you added, what was unclear,
and any names that break the standard.
```

2. Ask the main agent: "Use table-documenter on every table in Sales.SemanticModel, in parallel, and summarise what was unclear." The hook from 2.5 runs check_names.py after every edit, the sub-agents' edits too.

3. Read the result as a git diff, table by table. Where a sub-agent said something was unclear, decide and add the answer to glossary.md.

DONE WHEN

check_names.py reports no problems, you've read every description in the diff, and the unclear points are answered in glossary.md.

Descriptions and display folders are the part that lives in TMDL. Power BI's own tools for preparing a model for Copilot, such as AI instructions and verified answers, are set up in Power BI and aren't covered here.

2.7

Let the skill learn

USE THIS WHEN

The skill works, and you want it to get better with every task instead of only when someone remembers to update it.

YOU NEED

The skill in git.

STEPS

1. Add a section to SKILL.md:

```
## After each task
List conventions you met that aren't in this skill, and mistakes
you corrected. Propose each as an addition to the right reference
file, with one example. Add agreed decisions to
references/decisions.md with the date and the reason.
Don't change this skill until I approve.
```

2. After a task, read the proposals. Approve the ones that are true for the team, not only for this model.

3. Let the agent make the approved changes, and commit them with a message that says what was learned.

DONE WHEN

The skill has a change in git that came from a proposal you approved, and decisions.md has its first entry.

CHAPTER 3

Look it up

Templates, what's standard and what isn't, the safety notes, and the words used in this guide.

3.1 The one-page card

3.2 What works where

3.3 Before you rely on it

3.4 Words

3.1

The one-page card

A skill's folder

```
.claude/skills/model-standards/
  SKILL.md              the header and instructions (required)
  references/
    dax-style.md        read for DAX tasks
    glossary.md         business words, read for documentation
    decisions.md        decisions with their reasons
  scripts/
    check_names.py      exact checks; the agent reads the result
```

SKILL.md template

```
---
name: model-standards
description: <What it covers>. Use when <the tasks, in the words
  people type>.
---
# <Title>
<Rules, each with an example.>
For <topic>, read references/<file>.md.

## Before you finish
Run scripts/<check>.py and fix what it reports.

## After each task
Propose additions to this skill. Don't change it until I approve.
```

Optional in Claude Code: add `context: fork` under the description to run the skill as a sub-agent (1.7).

Hook settings (Claude Code)

```
"PreToolUse": [{
  "matcher": "mcp__<server>__.*",
  "hooks": [{ "type": "command", "command": "<guardrail>" }] }],
"PostToolUse": [{
  "matcher": "Edit|Write",
  "hooks": [{ "type": "command", "command": "<check>" }] }]
```

A hook command that exits with code 2 blocks the call (before) or shows its message to the agent (after).

3.2

What works where

The skill folder is an open standard. Hooks, sub-agents and running a skill as a sub-agent belong to the harness, and differ between harnesses.

PIECE	DEFINED BY	IN CLAUDE CODE
Name and description	Open standard	The SKILL.md header
Instructions, references, scripts	Open standard	The skill's folder
Calling a skill by name	The harness	`/model-standards`
Hooks	The harness	`.claude/settings.json`; a skill can also bring its own hooks in its header
Sub-agents	The harness	`.claude/agents/<name>.md`, with `skills:` to give it skills
A skill run as a sub-agent	The harness	`context: fork` in the skill's header
Learning loop	Instructions only	Works in any harness

Check your harness's documentation for the current names; the harness-specific parts change faster than the standard.

3.3

Before you rely on it

- **A skill's scripts run on your computer with your access.** Read a skill before you install it, the scripts especially, the way you'd read a macro from a stranger.
- **Instructions guide, hooks enforce.** If breaking a rule would cost real money or data, make it a hook.
- **Work on a copy.** Let the agent change the PBIP files or a development model, review the git diff, and publish the way you always do.
- **The agent can be confidently wrong.** A description that reads well can still describe the wrong thing. Read what goes in, especially before Copilot users rely on it.
- **MCP servers for Power BI are young.** Check which operations yours supports, and whether it's still in preview, before building a workflow on it.

3.4

Words

Agent harness (harness)	The app you chat with the agent in, which runs the model, reads the skills and calls the tools, such as Claude Code, GitHub Copilot in VS Code or Cursor. It runs the hooks.
Coding agent	An AI assistant that reads files, runs commands and calls tools.
Description	The line in a skill's header that says what it's for and when to use it. Always visible to the agent (1.3).
Display folder	A folder that groups columns and measures in a table's field list.
Hook	A command the harness runs automatically before or after an action. The agent can't skip it (1.6).
MCP server	A program that gives the agent tools, through the open Model Context Protocol. A Power BI MCP server gives tools for semantic models.
PBIP	Power BI project: a report and its semantic model saved as a folder of text files.
Reference file	A file in a skill's `references` folder, read only when a task needs it (1.4).
Script	A small program in a skill's `scripts` folder that does an exact check; the agent reads only its result (1.5).
Skill	A folder with a SKILL.md file of instructions that the agent reads when a task matches its description.
SKILL.md	A skill's one required file: a header with name and description, then the instructions.
Sub-agent	A second agent with its own, empty working memory, given one job by the main agent (1.7).
TMDL	The text format of a semantic model in a PBIP project, one file per table.
Working memory	Everything the agent has read in the current conversation. Technically, the context window.